

BACK-END DEVELOPMENT

WEEK 3 – Express Basics + POST + Middleware



After Finishing This Lecture:

- Create a basic HTTP server using Express.js
- Route HTTP requests using Express.js method
- Extract query parameters and request body using `express.json`
- Understand how Express request and response objects work

What is Express.js

- Express.js is a minimal and flexible Node.js web application framework
- Provides essential features for building web.
- Simplifies the creation of server-side applications through its abstractions and unobtrusive interface.



```
$ npm install express
```

Why Express?

- **Minimalist Web Framework:** Express.js is designed to be minimal yet powerful, providing just the essential building blocks. This allows developers to start small and scale complexity as needed without unnecessary bloat.
- **Abstracts Boilerplate:** Express simplifies the repetitive tasks associated with Node.js development by abstracting boilerplate code, allowing developers to focus on application logic rather than infrastructure setup.

Why Express?

- **Routing:** One of the standout features of Express is its routing capabilities, which enable you to define routes for different HTTP methods and URLs, thereby structuring your application logically and intuitively.
- **Middleware:** The concept of middleware is central to Express, empowering developers to process requests and responses in a flexible way. Middleware functions can perform a range of tasks from logging, authentication to serving static files.

Why Express? (Cont)

- **JSON Parsing:** Express comes built-in with middleware functionalities such as `express.json()`, which parses incoming requests with JSON payloads, facilitating easier manipulation of request data.
- **Static Files:** Serving static files such as images, CSS, and JavaScript is made easy through the `express.static()` middleware, allowing for straightforward integration of frontend assets into your application.

Why Express? (Cont)

- **URL Parsing and HTTP Headers:** Express streamlines handling URLs and HTTP headers, making it simple to access query parameters and header information within a request, enhancing overall application robustness.

Setting Up Express

Activity

- How is it similar to native HTTP module?
- How is it abstracted comparing to HTTP module?



```
npm init -y // Create node.js project  
npm install express
```



```
const express = require('express');  
const app = express();  
app.get('/', (req, res) => res.send("Hello World"));  
app.listen(3000, () => console.log('Server running'))
```




```
app.get('/', (req, res) => res.send('Welcome'));  
app.post('/submit', (req, res) => res.send('Received'));
```

Defining Routes

- app.get(path, handler) for reading
- app.post(path, handler) for creating
- req and res objects
- **Questions**
- How about other HTTP methods? (Delete, Put, Patch)



```
app.use(express.json());
app.post('/user', (req, res) => {
  console.log(req.body);
  res.send('User created');
});
```

Requires express.json() middleware

Handling POST Data

Request and Response Objects

The req object represents the incoming HTTP request. It contains:

`req.params` → route parameters

`req.query` → query string parameters

`req.body` → body of POST/PUT requests (after `express.json()`)

`req.method` → HTTP method (GET, POST, etc.)

`req.url` → full request path

Request and Response Objects (Cont)

The res object is used to send a response back to the client:

`res.send()` → send plain text, HTML, or JSON

`res.json()` → send a JSON object

`res.status()` → set HTTP status code

`res.redirect()` → redirect to another route

Dynamic & Query Params

`/users/:id` → `req.params.id`

`/users?id=12` → `req.query.id`

`/users/45` → `req.params.id` = ??

`/users?id=24` → `req.query.id` = ??

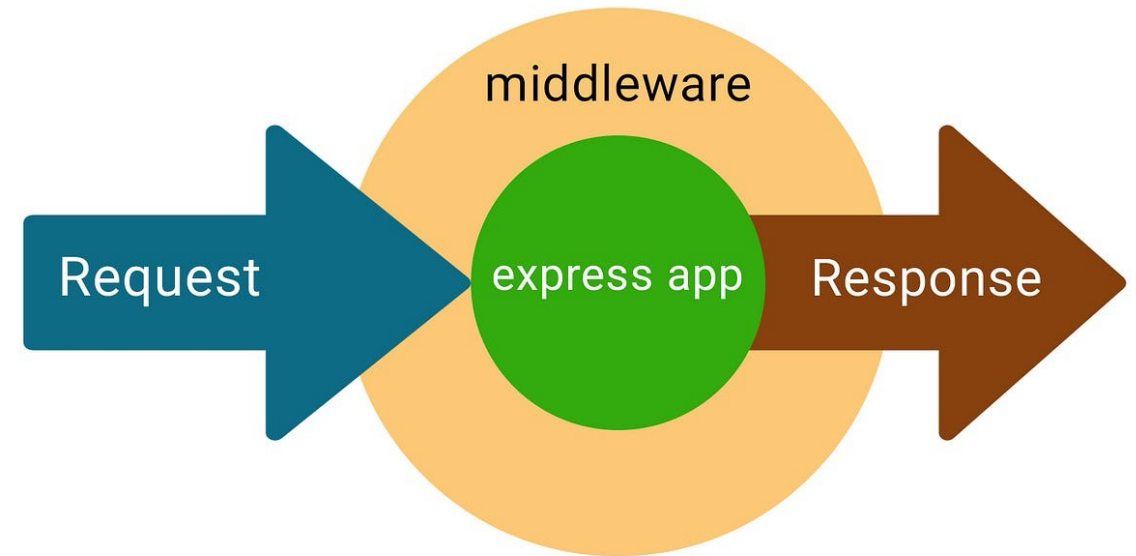
What is Middleware?

Middleware functions are functions that run **before** your route handler.

They can:

- Modify the req and res objects
- End the request-response cycle (res.send, etc.)
- Call the next middleware using next()

express middlewares



What is Middleware?

Imagine entering an airport:



You (the request) walk in



Middleware 1 = Security Check

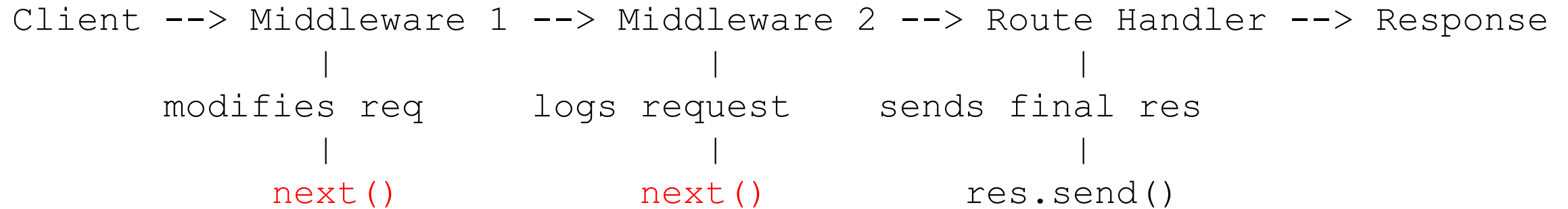


Middleware 2 = Passport Control



Route Handler = Boarding Gate

Request-Response Lifecycle Visualization



Basic Middleware Example

Question:

- What happens if we **don't call next()**?
- Where would you put middleware that checks for auth?

```
// Logger middleware
function logger(req, res, next) {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next(); // Important!
}

app.use(logger); // applies to ALL routes

app.get('/hello', (req, res) => {
  res.send('Hello World');
});
```

Type	Example
Application	<code>app.use(logger)</code>
Router-level	<code>router.use(authCheck)</code>
Built-in	<code>express.json()</code>
Error-handling	<code>(err, req, res, next) =></code>
Third-party	<code>morgan, cors, etc.</code>

Types of Middleware

Mini Activity

Try it yourself!

What would be logged and returned for:
`/products/42?category=books`



```
app.get('/products/:id', (req, res) => {  
  console.log(req.params); // ?  
  console.log(req.query);  // ?  
  res.send("Check the console");  
});
```

Mini Activity : Answer

```
req.params = { id: '42' }  
req.query = { category: 'books' }
```



WHAT WE HAVE LEARNT



- Create a basic HTTP server using Express.js
- Route HTTP requests using Express.js method
- Extract query parameters and request body using `express.json`
- Understand how Express request and response objects work